Model Accuracy Report — does the model agree with the ground truth?

This report answers three questions in plain terms:

1. After training, how often does the model's prediction match the ground truth, and how many predictions are correct versus incorrect?
2. The same question for **singletons** (Source-1 entities that have no true match at all).
3. Which algorithms are used, and what do they actually do?

It is for understanding only. Every number below comes from the **validation half of the dev split**, because that is the only data where we have ground truth *and* the model did not train on it. The test set has no labels, so nobody (including the organisers' leaderboard, until it is scored) can compute these numbers on it.

All numbers were produced by `code/business_entity_resolution/src/ber/val_report.py` (run it to regenerate them).

---

1. What exactly was measured

item	value
data scored	validation half of the dev split (S1 entities chosen by a hash of their row id)
S1 entities scored	**275,857**
true (S1, S2/S3) match pairs among them	**955,122**
candidate pairs the model looked at for these entities	17,177,876
model	the two-stage model trained on the *other* half of the dev split (`work/models/stage1_split.txt`, `stage2_split.txt`)
decision rule	one-parent filter + expected-F0.5 prefix (the same rule used for the submission)

Why not measure the final submitted models? They are trained on **all** dev entities, so scoring them on dev would be scoring on their own training data and would look better than

reality. The split model uses the identical recipe with half the data; the final models can only be equal or slightly better.

Vocabulary used below

- A **prediction** for an entity is the list of S2/S3 ids the model outputs (possibly empty).
- A prediction is **exactly correct** when the list equals the ground-truth list, including the case "empty list, and the entity truly has no match".
- At the pair level: **TP** = a predicted pair that is true, **FP** = a predicted pair that is false (a wrong merge), **FN** = a true pair the model did not output (a missed match).

---

2. Headline: entity-level agreement with the ground truth

	count	share of 275,857
Exactly correct entities (prediction list == ground-truth list)	**252,276**	**91.45 %**
Entities with at least one mistake	23,581	8.55 %
… of which: only missed matches (nothing wrong added)	21,051	7.63 %
… of which: only wrong matches added (nothing missed)	2,293	0.83 %
… of which: both missed and wrong	237	0.09 %
Entities scoring exactly 0 (every prediction wrong, or a wrong match on a singleton)	638	0.23 %
Macro-averaged F0.5 (the competition metric)	**0.9901**	

How the per-entity scores are distributed:

per-entity F0.5	entities
= 1.0 (perfect)	252,276
0.8 – 1.0 (one match missed out of several)	21,284
0.5 – 0.8	1,651
0 – 0.5	8
= 0	638

Reading this: when the model is wrong it is almost always *incomplete* rather than *wrong*.
An entity with 4 true matches where we return 3 scores 0.94; an entity where we add one wrong

match to 3 correct ones scores 0.79. Mistakes cluster in the first kind.

Did the model predict too few, too many, or the right number of matches?

relation	entities
predicted the same number of matches as the truth	252,498
predicted fewer than the truth	21,061
predicted more than the truth	2,298

---

3. Pair-level counts (every individual match decision)

	count
true match pairs in the ground truth	955,122
pairs the model output	934,666
correct pairs (TP)	**932,087**
wrong pairs output (FP, false merges)	**2,579**
true pairs missed (FN)	**23,035**
precision = TP / output	**99.72 %**
recall = TP / true	**97.59 %**
pair-level (micro) F0.5	0.9929

So out of every 1,000 matches the model outputs, about 997 are right and 3 are wrong; out of every 1,000 true matches, it finds about 976 and misses 24.

Where the 23,035 misses come from:

cause	pairs
the true partner was never proposed as a candidate (blocking miss)	4,778 (20.7 %)
the candidate existed but the model rejected it, and the S2/S3 record has a **blank address**	11,658 (50.6 %)
the candidate existed but the model rejected it, address present	6,599 (28.6 %)

Where the 2,579 wrong merges come from:

cause	pairs
S2/S3 record with an address (near-clone distractors: same name, house number off by a little)	1,975
S2/S3 record with a blank address (name equals a different entity with the same name)	604

Confusion matrix over all 17,177,876 candidate pairs the model scored for these entities:

	model said "match"	**model said "no match"**
truly a match	932,087 (TP)	18,257 (FN among candidates)
truly not a match	2,579 (FP)	16,224,953 (TN)

Note that 950,344 of the 955,122 true pairs were among the candidates (99.50 % blocking recall); the other 4,778 could never have been found by the model because they were not shown
to it.

---

4. Singletons: entities with no true match

A singleton scores 1.0 if we return an empty list and 0.0 if we return anything.

	count	**share**
singleton S1 entities in the validation set	**15,348**	5.56 % of all entities
correctly left empty	**15,182**	**98.92 %**
wrongly given one or more matches	166	1.08 %
… given exactly 1 wrong match	158	
… given 2 wrong matches	7	
… given 3 wrong matches	1	
total wrong pairs produced on singletons	175	
mean F0.5 over singletons	0.9892	

By country:

country	singletons	correct	share
US	9,272	9,168	98.88 %
India	6,076	6,014	98.98 %

What the 166 mistakes look like: the S2/S3 record is a "clone" of the singleton entity, most often a blank-address record whose name is identical to the entity's, or a record with the same name and a house number a few digits away. The model's probability for these is high because, from the strings alone, they look exactly like a genuine match. This is the irreducible part of the noise in this data.

---

5. Non-singletons: entities with at least one true match

	count	share of 260,509
entities with at least one true match	260,509	
exactly correct (all matches found, nothing wrong added)	**237,094**	**91.01 %**
all true matches found (possibly with an extra wrong one)	239,221	91.83 %
no wrong match added (possibly with a miss)	258,145	99.09 %
wrongly predicted as a singleton (returned an empty list)	468	0.18 %
mean F0.5	0.9902	

Broken down by how many true matches the entity has:

true matches	entities	exactly correct	share exact	mean F0.5	wrong pairs (FP)	missed pairs (FN)
1	14,824	14,199	95.8 %	0.9658	226	410
2	47,171	44,797	95.0 %	0.9891	552	1,918
3	66,169	61,368	92.7 %	0.9916	612	4,417
4	60,458	54,500	90.1 %	0.9922	510	5,874
5	40,351	35,544	88.1 %	0.9927	311	4,997
6	20,505	17,520	85.4 %	0.9928	138	3,241
7	8,101	6,817	84.1 %	0.9933	37	1,482
8	2,323	1,881	81.0 %	0.9928	14	527

true matches	entities	exactly correct	share exact	mean F0.5	wrong pairs (FP)	missed pairs (FN)
9	533	411	77.1 %	0.9924	3	148
10	70	56	80.0 %	0.9933	1	17
11	4	1	25.0 %	0.9796	0	4

Two things stand out. The share of *exactly* correct entities falls as the number of true matches grows (more chances to miss one), but the mean F0.5 *rises*, because one miss out of seven costs very little under F0.5. And entities with exactly one true match have the lowest mean score (0.9658): with a single match, one mistake is the whole entity.

By country:

country	entities	macro F0.5	exactly correct	share
US	165,945	0.9907	152,621	91.97 %
India	109,912	0.9892	99,655	90.67 %

---

6. What each part of the model contributed

The same validation entities scored with three configurations:

configuration	macro F0.5	exactly correct entities	wrong pairs (FP)	missed pairs (FN)	singletons correct
stage 1 only + expected-F0.5 rule	0.9864	245,123	3,890	30,172	14,832 (96.6 %)
stage 1 + stage 2, fixed threshold 0.5	0.9885	253,691	6,245	17,711	15,184 (98.9 %)
stage 1 + stage 2 + expected-F0.5 rule (used)	**0.9901**	252,276	**2,579**	23,035	15,182 (98.9 %)

Read across the rows:

- Adding **stage 2** (the context-aware second model) cut wrong merges by a third and fixed most singleton errors (516 → 166), because it can see when an S2/S3 record has a

better parent elsewhere.
- Switching from a plain **0.5 threshold** to the **expected-F0.5 rule** trades some recall for precision: 1,300 fewer exactly-correct entities but 3,666 fewer wrong merges. Under F0.5
  a wrong merge costs four times a miss, so the trade is worth +0.0016 in the metric. A threshold of 0.7 gets close (0.9899) but has to be tuned; the expected-F rule needs no tuning.

Are the model's probabilities honest? If the model says "70 % chance this is a match", is it right 70 % of the time? Yes, closely:

predicted probability	pairs	actually a match
0.0 – 0.1	16,196,647	0.03 %
0.1 – 0.2	19,007	12.6 %
0.2 – 0.3	8,475	21.5 %
0.3 – 0.4	5,399	30.8 %
0.4 – 0.5	4,339	41.0 %
0.5 – 0.6	3,685	49.4 %
0.6 – 0.7	3,646	61.1 %
0.7 – 0.8	3,924	73.0 %
0.8 – 0.9	5,758	84.8 %
0.9 – 1.0	926,996	99.87 %

This calibration is what makes the expected-F0.5 decision rule work: it only makes sense to compute an expected score if the probabilities mean what they say.

---

7. The algorithms, explained

The system is not one algorithm but a chain of six. Each is described below with what it does, why it was chosen, and the settings we used.

7.1 TF-IDF vectors + cosine similarity + sparse top-k search (blocking)

Problem it solves. 1.7 M reference records × 10 M other records is 1.7×10^{13} possible pairs; we can afford to look closely at about 10^{8}. We need a cheap way to find, for each record, the few dozen records that *might* match.

How it works.

- Every record becomes a *document*: the normalised name and address, as single words ("first", "food", "nashik") and as adjacent word pairs, called bigrams ("first_food", "food_private", "rue_fulton").
- **TF-IDF** (term frequency × inverse document frequency) turns a document into a sparse vector with one dimension per token. A token's weight is high when it is rare across the corpus (idf) and present in the document (tf, with a log-damped "sublinear" count). Rare, identifying tokens such as "first_food" or "gulmohor" get large weights; ubiquitous ones such as "private" or "road" get tiny weights. Tokens present in more than 5 % of documents are dropped entirely (`max_df = 0.05`), which is what keeps the search fast.
- **Cosine similarity** between two vectors is their dot product after each is scaled to length 1. It is 1.0 for identical token distributions and 0 for no shared tokens, and it is insensitive to word order and to the number of words, which suits shuffled and truncated records.
- **Top-k search** is a sparse matrix product (`sparse_dot_topn` , C++, multi-threaded): for each row on one side, multiply against all columns on the other side and keep the k largest cosines. We run it in both directions (each S2/S3 record finds its top 10 S1s; each S1 finds its top 20 S2/S3 records), plus an address-only pass and a name-only pass for records with blank addresses. The union is the candidate set.

Why this and not something fancier. It is unsupervised, needs no GPU, tolerates typos partially (a typo breaks one token but leaves the others), and its cost is controllable through `max_df` . Character n-grams (substrings of 3 letters) were tried for names; they are more typo-tolerant but far slower, so they are used only for the blank-address pass.

7.2 String similarity measures (features)

For every candidate pair we compute about 74 numbers describing how similar the two records are. The main string measures, all from the RapidFuzz library (C++):

- **Levenshtein distance**: the minimum number of single-character insertions, deletions or substitutions to turn one string into the other. "Nati0nal" → "National" is 1. We use it normalised (0–1) on names and raw on house-number digit strings.
- **Jaro-Winkler similarity**: counts matching characters within a sliding window and transpositions, with a bonus for a shared prefix. Good for short strings with typos.
- **ratio**: 1 − (Levenshtein-style edit distance on the whole string) / total length.
- **partial_ratio**: the best `ratio` of the shorter string against any equally long window of the longer one; catches "Peridance" inside "Peridance Peridance Aclhnmmy".
- **token_sort_ratio**: sort the words alphabetically on both sides, then `ratio` ; word order stops mattering ("& dixon brown, garcia" ≈ "Brown, Dixon & Garcia").

- **token_set_ratio**: compares the intersection of word sets with each side's remainder; robust to extra words on one side ("Brown, Dixon & Garcia **LP**").
- **Jaccard**: |A ∩ B| / |A ∪ B| on word sets.

Plus structural comparisons: is the first number in each address the same, what is the edit distance between the two numbers, is one a prefix of the other, do the sets of numbers conflict, does the last word (state/city stand-in) agree, how many S1 entities share this exact name, is the name a domain form, and so on. These are the features the classifier learns from; the classifier decides how to weigh them.

7.3 LightGBM: gradient-boosted decision trees (the classifier)

What it is. LightGBM is an implementation of *gradient boosting* over *decision trees*. It is the standard workhorse for tabular data (rows of numeric features), and is what wins most tabular competitions.

A decision tree asks a sequence of yes/no questions about the features ("is house-number edit distance ≤ 1?", "is name token-set ratio > 88?", "is the address blank?") and ends in a leaf with a score. One tree is crude.

Boosting builds trees one after another. Each new tree is fitted to the *errors* of all trees so far (technically, to the gradient of the loss with respect to the current predictions), and its output is added to the running total with a small step (the *learning rate*). After a few hundred to a few thousand trees the ensemble is a very flexible function of the features. The loss is the logistic loss, so the summed score passes through a sigmoid and becomes a probability between 0 and 1.

What makes LightGBM fast. Features are pre-binned into at most 255 histogram buckets; trees are grown *leaf-wise* (always splitting the leaf that reduces loss most, rather than level by level); and everything is multi-threaded.

Settings we used.

parameter	value	meaning
objective	binary	logistic loss, outputs a probability
num_leaves	127	maximum leaves per tree (model capacity)
min_data_in_leaf	100	a leaf must cover ≥ 100 pairs (prevents memorising noise)
learning_rate	0.05 (validation run) / 0.08 (final)	step size per tree

parameter	value	meaning
rounds	early-stopped at ≈ 1,000–1,500 (stage 1) and ≈ 250 (stage 2); fixed 900 / 250 for the final models	number of trees
feature_fraction, bagging_fraction	0.8	each tree sees a random 80 % of features and rows (regularisation)
lambda_l2	1.0	L2 penalty on leaf values
sample weights	1 for positives and hard negatives, 10 for the sampled easy negatives	keeps probabilities calibrated after subsampling

Why LightGBM and not a neural network. The inputs are ~74 engineered numbers, not raw text; tree ensembles handle such heterogeneous features, missing values (−1 flags), and interactions ("address blank AND name unique") natively, train in minutes on a CPU, and give calibrated probabilities. A neural model would need a GPU we do not have and would be harder to keep within the challenge's rules.

Training data. From the 34 M candidate pairs of the dev split we keep every positive pair, every *hard* negative (a negative whose name/address overlap is plausible, or whose address is blank) and a random 10 % of the easy negatives. That focuses the trees on the decisions that matter and keeps memory manageable; the weights undo the sampling bias so the
model still knows that most candidates are negatives.

7.4 Two-stage stacking with out-of-fold scores

Idea. The first LightGBM judges each pair in isolation. But the data has structure the pair cannot see: each S2/S3 record belongs to at most one S1 entity. So after stage 1 we compute, for every pair, 16 *context* numbers: how does this pair's score rank among all S1 candidates of the same S2/S3 record (is there a better parent?), what is the gap to the best, how many candidates of this S1 are already confident, and so on. A second LightGBM is trained on the original features plus these 16. Its most important features are the S2/S3-side margin and rank: "is this S1 clearly the best parent for this record?".

Out-of-fold. If stage 2 were trained on stage-1 scores produced by a model that had already seen those same pairs, the scores would be over-confident during training and ordinary at test time, and stage 2 would learn the wrong calibration. So the training entities are split into 3 folds; three stage-1 models are trained on two folds each and score the third. Every training pair's `s0` comes from a model that never saw it. A fourth stage-1 model, trained on everything, scores the test set.

7.5 The decision rule: expected-F0.5 prefix + one-parent filter

Why not a threshold. F0.5 is computed per entity. Adding a match with probability p is worth it only if the expected gain beats the expected loss, and that depends on how many matches the entity already has. Worked out from the formula, the break-even probability is about 0.50 for an entity's first match, 0.73 for a second, 0.77 for a fourth. A single global threshold cannot express that.

What we do instead. For each S1 entity:

1. **One-parent filter** — an S2/S3 record that appears as a candidate for several S1 entities is kept only for the one where its probability is highest (each record has at most one parent).
2. Sort the remaining candidates by probability $p_1 \geq p_2 \geq \dots$.
3. For each prefix length k = 0, 1, 2, … (up to 12), compute the **expected F0.5** if we output exactly the top k, treating each candidate as an independent coin with probability p_i. The number of true positives inside the prefix and the number of misses outside it are both *Poisson-binomial* random variables (sums of independent but unequal coins); their distributions are computed exactly with a small dynamic programme, and the expectation of 1.25·TP / (1.25·TP + 0.25·FN + FP) is summed over all outcomes.
4. Output the prefix with the highest expectation, which may be the empty list.

On validation this scored 0.9901 versus 0.9885 for a 0.5 threshold and 0.9899 for the best threshold found by search (0.7).

7.6 Learned transliteration dictionary

About 14 % of India S2 names are written in an Indic script (Devanagari, Telugu, Tamil, …) while the S1 name is Latin. Generic transliteration (`unidecode`) produces near-garbage for these. Instead, from every matched training pair whose S2/S3 field contains foreign-script tokens, we count which Latin token appears at the same position in the S1 field (when the word counts agree) or anywhere in it (otherwise). The most frequent partner becomes the dictionary entry: सुपर → super, प्रॉपर्टीज → properties, ಕರ್ನಾಟಕ → karnataka. 1,363 entries cover 89 % of the 1,534 distinct foreign tokens; the rest fall back to `unidecode` . This is simple co-occurrence counting, not a neural model, and it is learned entirely from the provided training data.

7.7 The metric, with a worked example

```
F0.5 = 1.25 · TP / (1.25 · TP + 0.25 · FN + FP)     per entity, then
averaged
```

entity truth	our prediction	TP	FP	FN	F0.5
{A, B, C}	{A, B, C}	3	0	0	1.000
{A, B, C}	{A, B}	2	0	1	2.5 / 2.75 = 0.909
{A, B, C}	{A, B, C, X}	3	1	0	3.75 / 4.75 = 0.789
{A}	{X}	0	1	1	0.000
{} (singleton)	{}	0	0	0	1.000 by definition
{} (singleton)	{X}	0	1	0	0.000

Missing one of three costs 0.09; adding one wrong one to three costs 0.21. That asymmetry is why the model is tuned to abstain when unsure, and why 91 % of our errors are omissions.

---

8. How the dataset was used: did we train on all of it?

No. The model was trained on a deliberately chosen **25 % slice** of the training data, and the whole test set was used at inference. The details:

8.1 What was taken from the training data

set	rows available	rows used for model training
Source 1 (train)	2,206,821	551,478 (a random 25 %)
Source 2 + 3 (train)	10,320,219	2,578,369 (all 1,908,432 true matches of those 551 k entities, plus a random 25 % of the 2.68 M records that match nothing)
ground-truth match rows	7,638,365	1,908,432

This slice is the "dev split" built by `make_dev.py`. It keeps the same shape as the test set: every entity comes with **all** of its true matches and with the same proportion of distractor records, so candidate counts, blank-address rates and near-clone density look like the real thing. The remaining 75 % of the training entities were used for one thing only, the transliteration dictionary (`learn_translit.py` reads all 2.2 M entities and their matches, because the dictionary is cheap to build and benefits from every example).

8.2 Why a slice and not everything

- The unit of learning is not a record but a **candidate pair** with a 0/1 label. 551 k entities already produce 34.25 M candidate pairs and 1.9 M positive pairs. The model learns patterns of string similarity, not individual businesses, and that many examples is already far past the point where a tree model keeps improving.
- Cost scales with rows. The 25 % slice took about 1 hour end to end (blocking 15 min, features 8 min, training 35 min). The full training set would have cost four times that, and blocking alone on the test set already took 4 hours on the 16 GB laptop. Using a 50 % slice is the first thing to try if more compute time is available (see the walkthrough, §15).

8.3 How 34 M candidate pairs became training rows

Even 34 M pairs do not fit in memory with 74 features each, so the feature file is streamed in 2 M-row slices and sampled:

what	validation run (half of the dev entities)	final model (all dev entities)
positive pairs	948,460 (all kept)	1,898,804 (all kept)
hard negatives (plausible overlap, or blank address)	4,063,970 (all kept)	about 8.1 M (all kept)
easy negatives	random 10 %, weighted ×10	random 10 %, weighted ×10
rows actually fed to LightGBM	6,219,987	12,477,415

Every pair is still *scored* by stage 1 (that is where the context features come from); only the fitting of the trees uses the sample. The ×10 weights make the sample statistically equivalent to the full set, which is why the probabilities stay calibrated (§6).

8.4 Validation inside the slice

The 551 k dev entities were split in half by a hash of their row id. One half trained the model; the other half (275,857 entities) was scored to produce every number in §§2–6. The submitted models were then retrained on all 551 k entities with the same settings.

8.5 The test set was used in full

All 1,732,544 test entities and all 9,969,589 test S2/S3 records went through blocking (126.7 M candidate pairs) and scoring. The unsupervised parts, such as the TF-IDF vocabularies, the "generic token" lists and the name-ambiguity counts, are computed from the test records themselves, which is also how France was handled without any French training data.

8.6 How the large row counts were kept manageable

- Everything runs one country at a time, because no match crosses a country.
- Blocking works on sparse TF-IDF matrices with frequent tokens pruned, in 50 k-row chunks.
- Features are computed in 500 k-pair chunks by worker processes and written to disk.
- Inference streams pairs from disk, keeps only a small per-pair frame (ids plus scores) for a whole country, and deletes each chunk once both models have scored it. Peak memory on the
  test set was 4.9 GB.

---

9. One-paragraph summary

On 275,857 held-out entities the model reproduces the ground truth exactly for 252,276 (91.45 %) and scores 0.9901 macro F0.5. At the level of individual matches it outputs 934,666 pairs of which 932,087 are correct (99.72 % precision) and misses 23,035 of the 955,122 true pairs (97.59 % recall). Singletons are handled well: 15,182 of 15,348 (98.92 %) are correctly left empty. Errors are dominated by omissions on blank-address records whose name is ambiguous, and by near-clone distractors that differ from the true entity only in a house number; both are limits of the information in the strings rather than of the learning algorithm. The algorithms are TF-IDF nearest-neighbour blocking, RapidFuzz string similarities, two stacked LightGBM gradient-boosted tree classifiers trained with out-of-fold scores and weighted hard-negative sampling, and an exact expected-F0.5 decision rule.